

武汉理工大学

课程报告

字段	内容
课程名称	企业级应用软件设计与开发
项目名称	扫地机器人智能客服 Agent 系统
方向	方向一：Agentic AI 原生开发
学号	2025303047
姓名	毛远旭
专业	软件工程
指导教师	戚欣
提交日期	2026 年 6 月 22 日

选题背景与设计思想

问题定义

近年来，扫地/扫拖一体机器人已从高端消费品逐步走入普通家庭。据行业数据，中国扫地机器人市场年销量已突破 500 万台，用户群体涵盖独居青年、养宠家庭、老年用户等多种类型。然而，随着设备功能的日益复杂（扫拖一体、自动集尘、基站自清洁、AI 避障等），用户在使用过程中面临的困惑和故障场景也显著增多。

典型用户问题包括：

- **选购决策**：“小户型适合买扫拖一体机吗？”
- **使用技巧**：“木地板用湿拖模式会不会鼓包？”
- **故障排查**：“机器人总被地毯卡住怎么办？”
- **维护保养**：“滤网多久换一次？边刷什么时候该换？”
- **环境适配**：“上海梅雨天开扫地机器人会发霉吗？”

现有方案的不足

当前市场上扫地机器人品牌的客服体系主要依赖以下方式：

现有方案	痛点
人工客服	7×24 不可用，响应慢，专业知识参差不齐
传统 FAQ 检索	关键词匹配，无法理解复杂语义，无法多步骤推理
说明书/App 帮助	信息静态，无法结合用户实际情况（如天气、历史记录）
通用大模型对话	缺乏垂直领域专业知识，可能产生幻觉或给出不安全建议

核心差距在于：**用户问题往往需要综合多源信息（产品知识 + 实时环境 + 个人使用数据）进行多步推理才能给出专业回答**，而传统方案缺乏这种能力。

项目价值

本项目的核心价值在于构建一个具备 **ReAct (Reasoning + Acting) 推理能力** 的智能客服 Agent，它能够：

1. **理解用户意图**：判断用户是在咨询、排查故障、还是需要个人报告
2. **自主决策工具调用**：根据需求自动选择知识库检索、天气查询、用户数据拉取等工具
3. **多源信息整合**：将检索到的专业知识、实时环境信息、用户个人数据整合为完整回答
4. **动态模式切换**：在普通问答模式和报告生成模式之间自动切换

技术路线

本项目采用 **Agentic AI 原生开发** 路线，核心技术栈：

- **推理框架**：LangGraph ReAct Agent — 实现「思考→行动→观察→再思考」的自主推理循环
- **知识检索**：Chroma 向量数据库 + RAG — 将扫地机器人专业知识文档向量化，支持语义检索
- **工具系统**：LangChain @tool 装饰器 + MCP 协议 — 将外部能力封装为标准化工具
- **中间件机制**：LangGraph Middleware — 实现工具调用监控、上下文注入、动态提示词切换
- **模型层**：阿里通义千问 qwen3-max (LLM) + DashScope text-embedding-v4 (嵌入)

整体技术路线遵循 **SDD (规格驱动开发)** 方法论：先定义 Product Spec → 设计 Architecture Spec → 编写 API Spec → 实现 → 测试验证。

Specs 规格文档

Product Spec（产品规格）

功能需求

编号	功能	描述	优先级
F1	专业知识问答	基于 RAG 检索知识库，回答扫地机器人的选购、使用、维护问题	P0
F2	天气环境分析	通过 MCP 获取城市天气，分析环境条件对机器人使用的影响	P1
F3	故障排查指导	根据用户描述的故障现象，给出逐步排查建议	P0
F4	个人使用报告	拉取用户使用记录，生成 Markdown 格式的个性化使用报告	P0
F5	多轮对话	支持多轮对话，保持上下文连贯性	P1
F6	流式输出	回答以流式方式逐字输出，提升用户体验	P1

用户场景

场景 A：普通知识问答 > 用户：“家里的扫地机器人总是在地毯上卡住，怎么办？” > Agent：思考 → 判断需要专业知识 → 调用 `rag_summarize("机器人 地毯 卡住 解决")` → 获取知识库内容 → 整合回答（建议检查地毯厚度、设置虚拟墙、调整越障模式等）

场景 B：环境关联问答 > 用户：“最近上海梅雨天，扫地机器人怎么用比较好？” > Agent：思考 → 需要天气 + 专业知识 → 并行调用 `get_weather("上海")` 和 `rag_summarize("梅雨天 扫地机器人 注意事项")` → 整合回答（建议减少湿拖、注意防潮、及时清理尘盒防霉等）

场景 C：个人使用报告 > 用户：“帮我查一下我这个月的机器人使用情况” > >
Agent：识别报告意图 → 调用 `fill_context_for_report` → 中间件切换提示词到报告模式 → 调用 `get_user_id()` → `get_current_month()` → `fetch_external_data(user_id, month)` → 生成 Markdown 报告

Architecture Spec（架构规格）

系统分层



核心设计决策

决策点	选择	理由
Agent 模式	ReAct	业界成熟的推理-行动循环，LangGraph 原生支持
工具定义方式	@tool 装饰器 + MCP	本地工具用 @tool，外部服务用 MCP 实现解耦
向量数据库	Chroma	轻量级，支持持久化，与 LangChain 深度集成
提示词管理	YAML + 独立 txt 文件	业务逻辑与提示词分离，便于迭代优化
模型层	抽象工厂模式	方便切换不同模型提供商，符合开闭原则

决策点	选择	理由
报告模式切换	中间件 + 动态提示词	不侵入主流程，通过中间件优雅实现模式切换

API Spec（工具接口规格）

工具一：rag_summarize

名称：

rag_summarize

描述：

从向量存储中检索参考资料

入参：

query：

string – 检索词，直接使用用户问题的核心关键词

出参：

string – 基于检索资料生成的总结性回答

示例：

入参：

"小户型 适合 扫地机器人"

出参：

"根据参考资料，小户型（40-60m²）推荐选择机身轻薄、续航30分钟以上的..."

工具二：get_weather（MCP）

名称：

get_weather

协议：

MCP（Model Context Protocol）

传输：

HTTP（localhost:8000/mcp）

描述：

联网搜索指定城市的实时天气信息

入参：

city：

string – 标准城市名称，如"上海"、"北京"

出参：

string – 天气标题、来源URL、内容摘要

工具三：get_user_id

名称：

get_user_id

描述：

获取当前用户的唯一标识

入参：

无

出参：

string – 用户ID（如"1002"）

工具四：get_current_month

名称：

get_current_month

描述：

获取系统当前月份

入参：

无

出参：

string – 月份，格式YYYY-MM（如"2025-06"）

工具五：fetch_external_data

名称：

fetch_external_data

描述：

获取指定用户指定月份的使用记录

入参：

user_id:

string – 用户ID（数字字符串）

month:

string – 月份，格式YYYY-MM

出参：

string – 结构化使用记录（特征/效率/耗材/对比）或空字符串

工具六：fill_context_for_report

名称：

fill_context_for_report

描述：

触发中间件为报告生成场景注入上下文，不返回数据

入参：

无

出参：

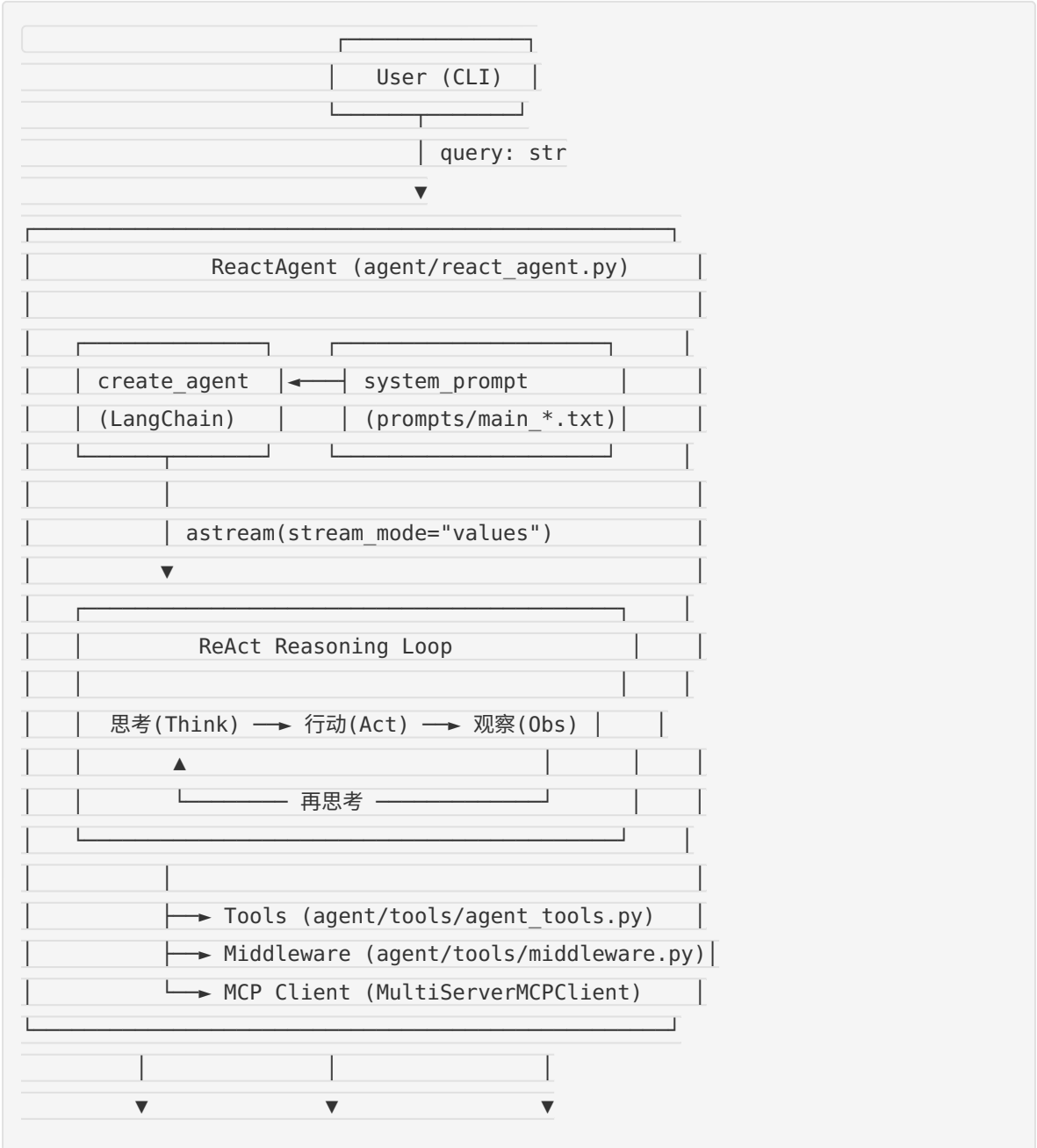
无（由中间件捕获并设置 context.report = True）

系统架构与设计

核心架构图

本系统采用分层架构设计，清晰分离关注点。以下从整体架构、Agent 交互流程、数据流三个维度展示系统设计。

系统整体架构



RAG Service	MCP Server	External Data
(Chroma DB)	(FastMCP)	(records.csv)

Agent 交互流程

```
graph TD
    A[用户输入问题] --> B{Agent 思考判断}
    B -->|需要专业知识| C[调用 rag_summarize]
    B -->|需要天气信息| D[调用 get_weather MCP]
    B -->|识别报告意图| E[调用 fill_context_for_report]
    B -->|信息足够| F[直接生成回答]

    C --> G[Chroma 向量检索]
    G --> H[RAG 总结生成]
    H --> B

    D --> I[MCP HTTP → Tavily 搜索]
    I --> B

    E --> J[中间件设置 context.report=true]
    J --> K[动态切换提示词→report_prompt.txt]
    K --> L[调用 get_user_id / get_current_month]
    L --> M[调用 fetch_external_data]
    M --> B

    B --> N{是否达到5次工具调用?}
    N -->|是| O[回复：我不知道]
    N -->|否| B

    F --> P[流式输出给用户]
    P --> Q[结束]
    O --> Q
```

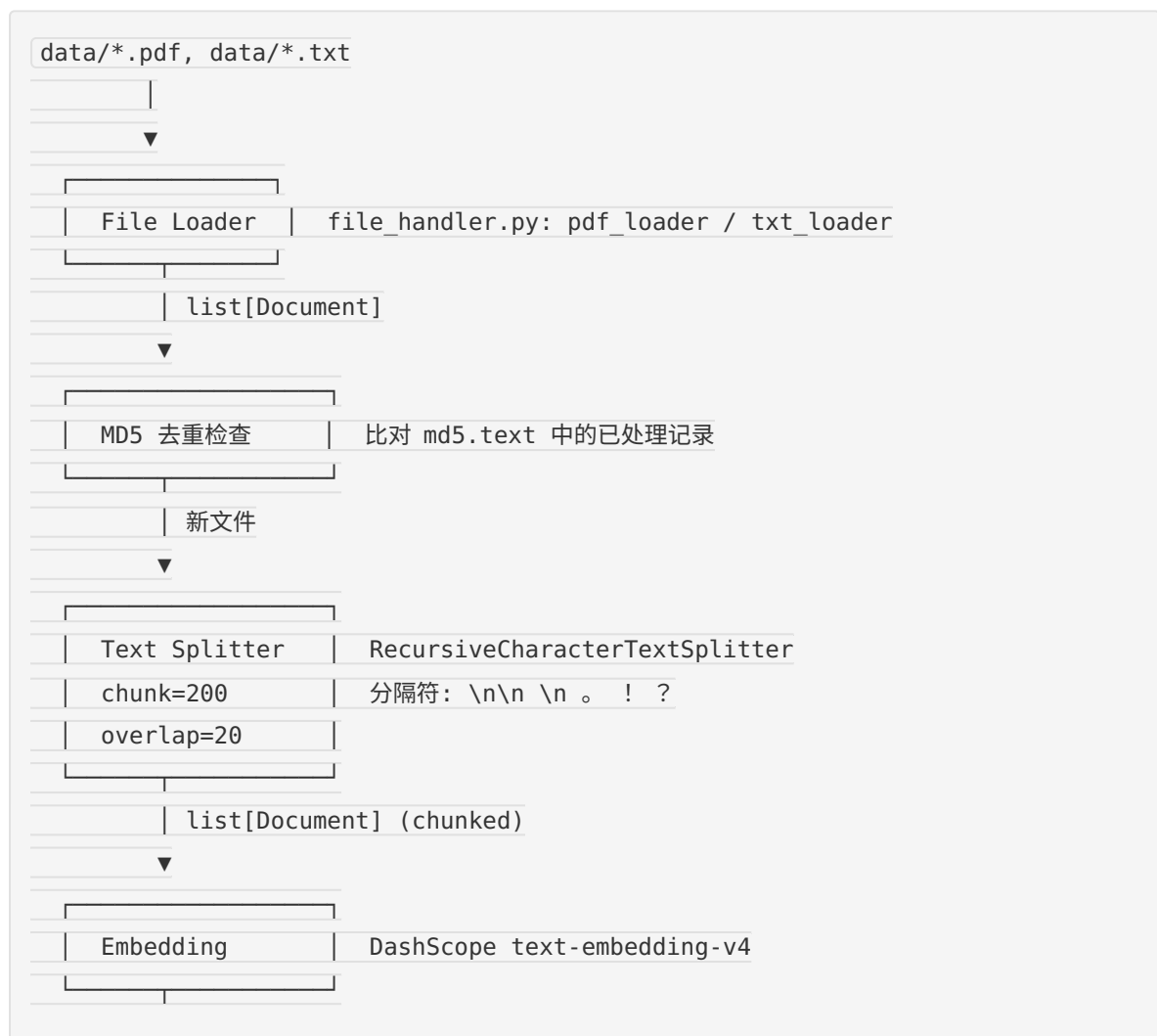
交互流程简述

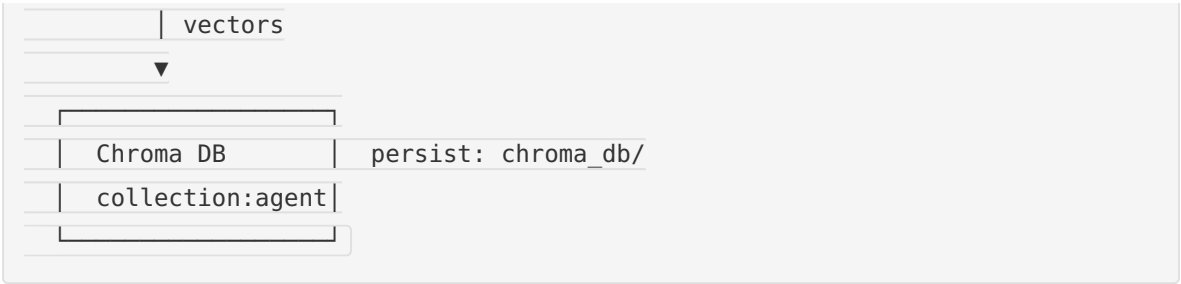
1. 用户输入阶段：用户通过 CLI 提交自然语言问题

2. **Agent 思考阶段**: LLM 根据系统提示词判断用户核心需求、确定所需信息是否充足
3. **工具调用阶段**: Agent 根据判断结果自主选择合适的工具（RAG 检索 / MCP 天气 / 用户数据等）
4. **中间件介入**: 每次工具调用被 `monitor_tool` 拦截记录；每次模型调用前被 `log_before_model` 记录；`report_prompt_switch` 根据上下文动态切换提示词
5. **观察再思考**: Agent 获取工具返回结果后重新判断信息是否充足，决定继续调工具或生成最终回答
6. **回答生成**: 通过 `astream` 流式输出，逐步返回给用户

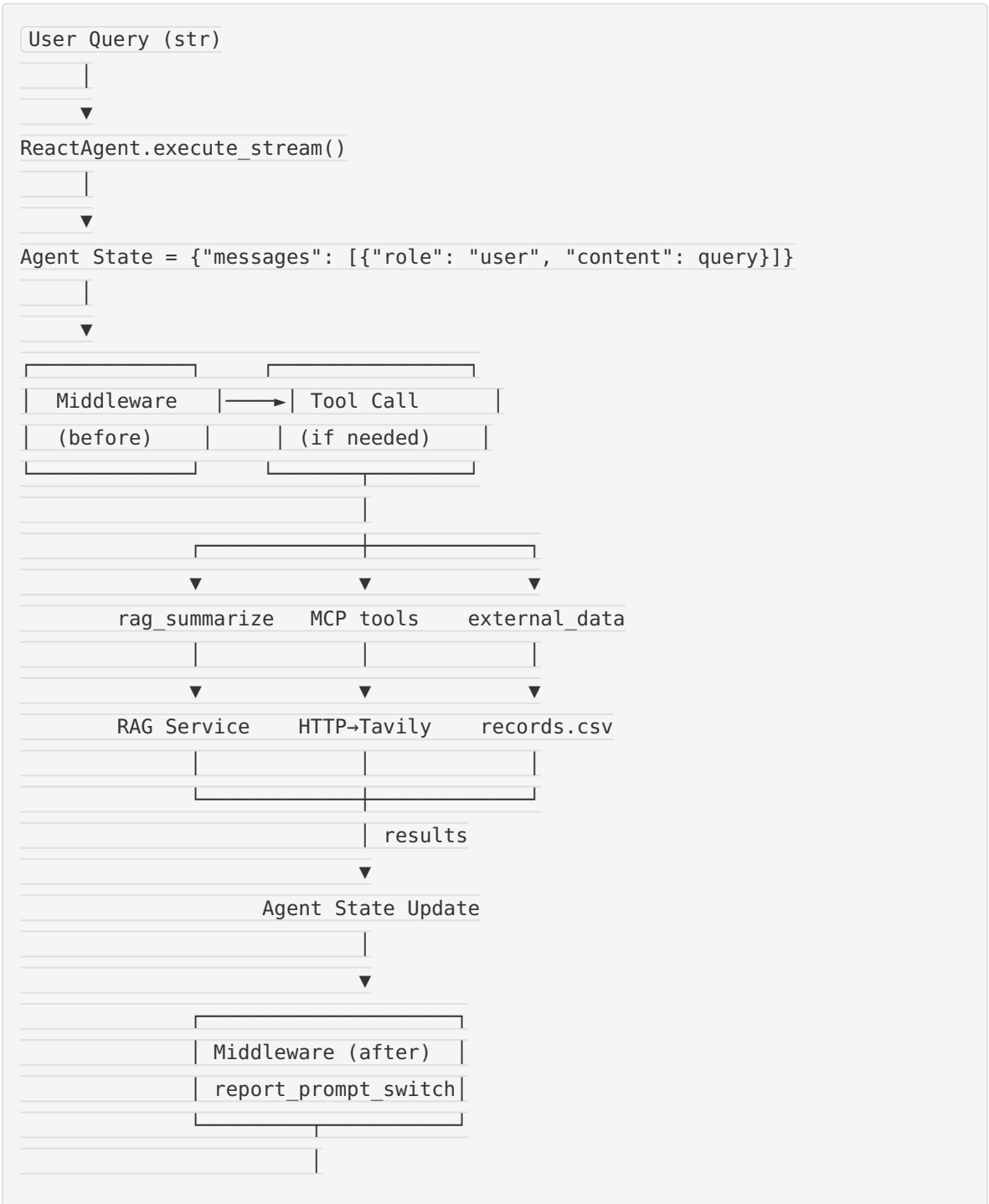
数据流设计

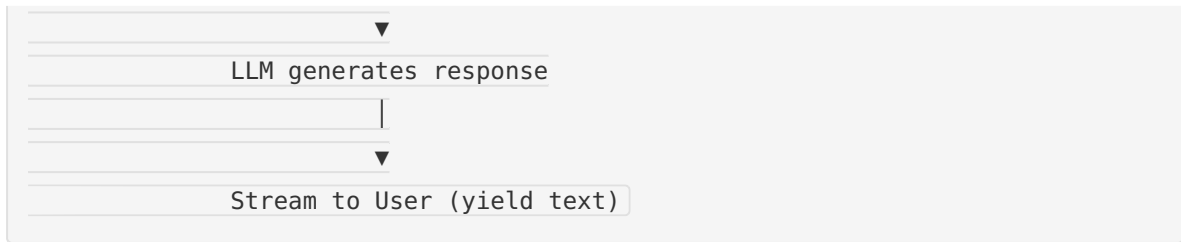
知识库数据流





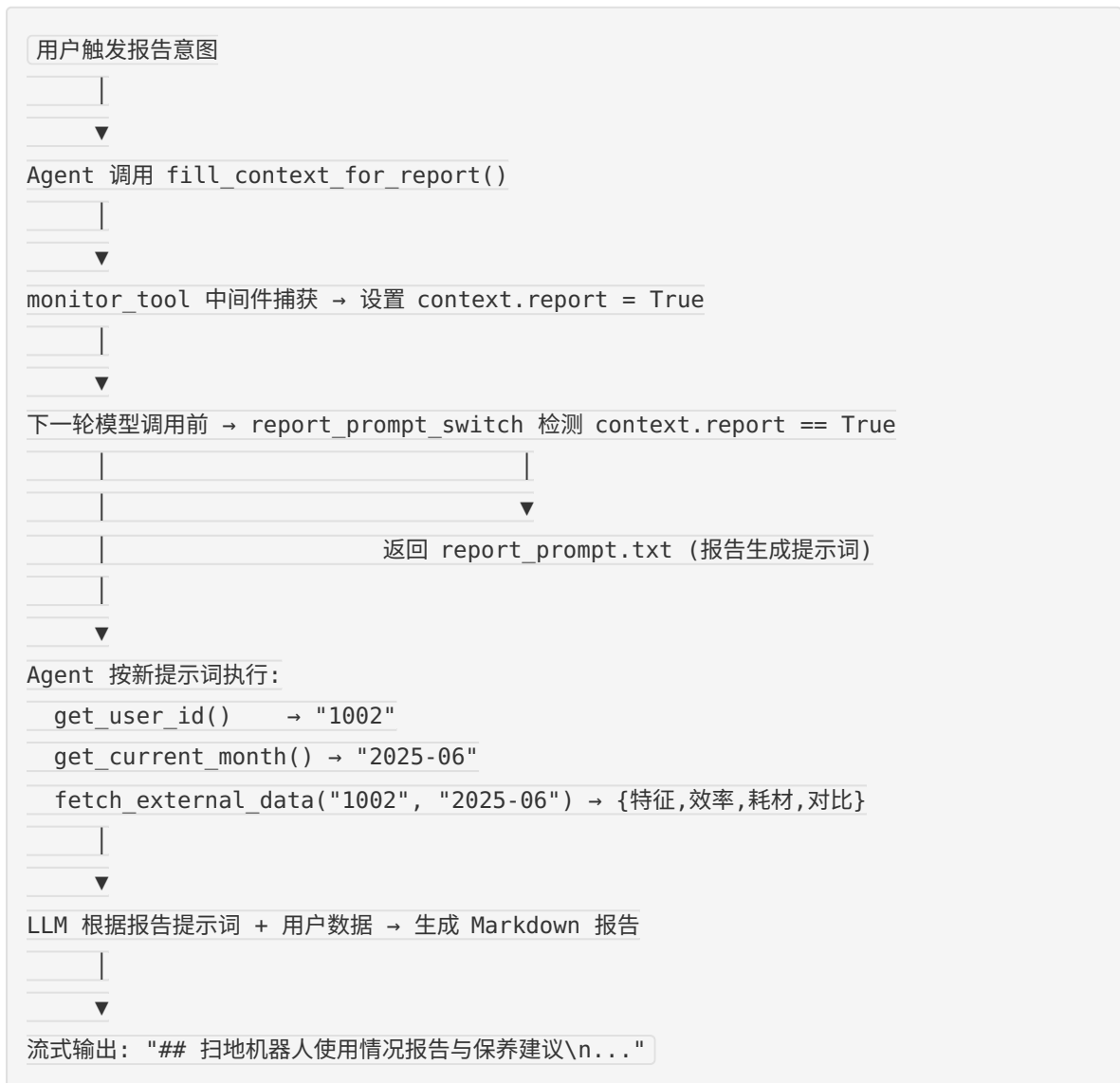
请求处理数据流





报告生成模式数据流

报告生成是本系统最具特色的功能，其数据流较普通问答更为复杂：



关键实现与代码展示

Agent 核心循环

Agent 的主入口为 `ReactAgent` 类，基于 LangChain 的 `create_agent` 构建，是系统最核心的组件。

创建流程 (`agent/react_agent.py`)

```
@classmethod
async def create(cls):
    # 1. 建立 MCP 客户端连接 (HTTP 传输)
    client = MultiServerMCPClient({
        "weather": {
            "transport": "http",
            "url": "http://localhost:8000/mcp",
        }
    })
    mcp_tools = await client.get_tools()

    # 2. 创建 Agent, 注册工具和中间件
    agent = create_agent(
        model=chat_model, # ChatTongyi(qwen3-max)
        system_prompt=load_system_prompts(), # main_prompt.txt
        tools=[
            rag_summarize, # RAG 知识检索
            get_user_location, # 获取用户位置
            get_user_id, # 获取用户ID
            get_current_month, # 获取当前月份
            fetch_external_data, # 拉取外部数据
            fill_context_for_report, # 上下文注入 (报告模式触发)
            *mcp_tools, # MCP 天气工具
        ],
        middleware=[monitor_tool, log_before_model, report_prompt_switch],
    )
    return cls(agent)
```

流式执行 (`agent/react_agent.py`)

```

async def execute_stream(self, query: str):
    input_dict = {
        "messages": [{"role": "user", "content": query}]
    }
    async for chunk in self.agent.astream(
        input_dict,
        stream_mode="values",
        context={"report": False}
    ):
        latest_message = chunk["messages"][-1]
        if latest_message.content:
            text = _normalize_content(latest_message.content)
            if text:
                yield text + "\n"

```

设计要点:

- 使用 `stream_mode="values"` 在每个状态更新时获取最新消息
- `context={"report": False}` 初始化报告模式上下文为关闭状态
- `_normalize_content()` 兼容 `str`、`list[dict]`、`list[str]` 等多种消息格式

工具定义 (`agent/tools/agent_tools.py`)

系统定义了 6 个工具函数，分为三类：RAG 检索类、用户数据类、上下文控制类。

RAG 检索工具

```

@tool(description="从向量存储中检索参考资料")
def rag_summarize(query: str) -> str:
    return rag.rag_summarize(query)

```

该工具封装了 `RagSummarizeService`，内部链路为：向量检索 → 上下文拼接 → LLM 总结生成。

用户信息工具

```
@tool(description="获取用户的ID，以纯字符串形式返回")
def get_user_id() -> str:
    return random.choice(user_ids)

@tool(description="获取当前月份，以纯字符串形式返回")
def get_current_month() -> str:
    return random.choice(month_arr)
```

注：当前阶段使用模拟数据。生产环境中，`get_user_id()` 应从认证层获取真实用户 ID，`get_current_month()` 应返回系统真实时间。

外部数据拉取工具

```
@tool(description="从外部系统中获取指定用户指定月份的使用记录")
def fetch_external_data(user_id: str, month: str) -> str:
    generate_external_data() # 延迟加载，首次调用时将 CSV 加载到内存
    try:
        return external_data[user_id][month]
    except KeyError:
        return ""
```

`generate_external_data()` 采用惰性加载模式——首次调用时解析 `records.csv`，构建嵌套字典 `{user_id: {month: {数据}}}`，后续调用直接内存查找，提升性能。

报告上下文注入工具

```
@tool(description="调用后触发中间件自动为报告生成场景注入上下文")
def fill_context_for_report():
    return "fill_context_for_report已调用"
```

这是一个**哨兵工具**——本身不返回有意义数据，其调用事件被中间件 `monitor_tool` 捕获，触发报告模式的上下文切换。这种设计将模式切换逻辑与 Agent 主循环解耦。

中间件机制 (`agent/tools/middleware.py`)

中间件是 Agent 的“隐形骨架”，在每次工具调用、每次模型推理前后自动执行。系统定义了三个中间件：

1. 工具调用监控 (`@wrap_tool_call`)

```
@wrap_tool_call
async def monitor_tool(request, handler):
    logger.info(f"[tool monitor]执行工具: {request.tool_call['name']}")
    logger.info(f"[tool monitor]传入参数: {request.tool_call['args']}")

    result = handler(request)

    if inspect.isawaitable(result):
        result = await result

    # 关键逻辑: 检测报告模式触发哨兵
    if request.tool_call['name'] == "fill_context_for_report":
        request.runtime.context["report"] = True

    return result
```

监控每个工具的执行情况（名称、参数、成功/失败），同时拦截

`fill_context_for_report` 调用以切换报告模式。

2. 模型调用前日志 (`@before_model`)

```
@before_model
def log_before_model(state, runtime):
    logger.info(f"[log_before_model]即将调用模型， "
                f"带有{len(state['messages'])}条消息。")

    return None
```

记录每轮模型调用的状态快照，便于调试和问题追踪。

3. 动态提示词切换 (`@dynamic_prompt`)

```
@dynamic_prompt
def report_prompt_switch(request: ModelRequest):
```

```

is_report = request.runtime.context.get("report", False)
if is_report:
    return load_report_prompts() # prompts/report_prompt.txt
return load_system_prompts()    # prompts/main_prompt.txt

```

每次模型调用前检查 `runtime.context["report"]`，动态决定使用通用提示词还是报告生成提示词。这是系统实现**模式自适应切换**的核心机制。

RAG 服务实现 (`rag/rag_service.py`)

```

class RagSummarizeService:
    def __init__(self):
        self.vector_store = VectorStoreService()
        self.retriever = self.vector_store.get_retriever()
        self.prompt_template = PromptTemplate.from_template(load_rag_prompts())
        self.model = chat_model
        self.chain = self.prompt_template | self.model | StrOutputParser()

    def rag_summarize(self, query: str):
        docs = self.retriever.invoke(query)
        documents = ""
        for i, doc in enumerate(docs, 1):
            documents += f"【参考资料{i}】: {doc.page_content}"
            f"| 元数据: {doc.metadata}\n"
        return self.chain.invoke({"input": query, "context": documents})

```

使用 LangChain LCEL (LangChain Expression Language) 的管道操作符 `|` 构建处理链：**提示词模板** → **LLM** → **字符串解析器**，代码简洁明了。

MCP 天气服务 (`agent/mcp/get_weather.py`)

```

from fastmcp import FastMCP
from langchain_tavily import TavilySearch

mcp = FastMCP("get_weather")
tavily = TavilySearch(max_results=1, topic="general")

@mcp.tool(description="联网搜索指定城市的天气")

```

```
def get_weather(city: str) -> str:
    resp = tavily.invoke({"query": f"{city} 天气"})
    results = resp.get("results") or []
    if not results:
        return "未找到相关天气信息"
    top = results[0]
    return f"{top['title']}\n{top['url']}\n{top['content']}".strip()

mcp.run(transport="http") # 在 localhost:8000/mcp 监听
```

将天气搜索能力封装为独立的 MCP HTTP 服务，实现了**工具提供方与 Agent 的解耦**。Agent 通过 `MultiServerMCPClient` 动态发现并加载该工具，无需重启 Agent 即可更新 MCP 服务。

模型工厂 (`model/factory.py`)

```
class BaseModelFactory(ABC):
    @abstractmethod
    def generator(self) -> Optional[Embeddings | BaseChatModel]:
        pass

class ChatModelFactory(BaseModelFactory):
    def generator(self):
        return ChatTongyi(model=rag_conf["chat_model_name"])

class EmbeddingsFactory(BaseModelFactory):
    def generator(self):
        return DashScopeEmbeddings(model=rag_conf["embedding_model_name"])

chat_model = ChatModelFactory().generator()
embed_model = EmbeddingsFactory().generator()
```

采用抽象工厂模式，如需切换到其他 LLM 提供商（如 DeepSeek、OpenAI），只需新增一个 Factory 子类并修改配置，无需改动业务代码。

配置文件结构

所有配置使用 YAML 格式，分离为 4 个文件：

配置文件	职责	核心字段
<code>config/rag.yml</code>	模型名称	<code>chat_model_name: qwen3-max</code> , <code>embedding_model_name: text-embedding-v4</code>
<code>config/chroma.yml</code>	向量库	集合名、分块大小、重叠量、分隔符、检索 top-k
<code>config/prompt.yml</code>	提示词路径	指向 <code>prompts/</code> 下的三个 txt 文件
<code>config/agent.yml</code>	Agent 参数	外部数据文件路径 <code>data/external/records.csv</code>

截图位置：此处可添加 AI IDE（VS Code + Claude Code）开发过程截图，展示代码编辑、Agent 对话、工具调用调试等界面。

测试与评估

功能测试

测试用例设计

用例编号	测试场景	输入	期望行为	覆盖工具
TC-01	选购咨询	“小户型适合什么扫地机器人”	调用 rag_summarize 检索选购资料并给出推荐	rag_summarize
TC-02	故障排查	“机器人一直提示请清理边刷”	调用 rag_summarize 检索故障处理方案	rag_summarize
TC-03	环境适配	“今天上海天气适合用扫地机吗”	调用 get_weather(“上海”) + rag_summarize	rag_summarize, MCP
TC-04	报告生成	“生成我的使用报告”	fill_context_for_report → get_user_id → get_current_month → fetch_external_data → Markdown 报告	全部工具
TC-05	工具调用上限	连续 6 次触发工具调	第 6 次时回复“我不知道”	全部工具

用例编号	测试场景	输入	期望行为	覆盖工具
		用的复杂问题		
TC-06	异常处理	查询不存在 的用户月份数据	fetch_external_data 返回空字符串，Agent 妥善告知用户	fetch_external_data
TC-07	多轮对话	先问选 购再追 问维护	保持上下文，第二轮回 答关联第一轮话题	rag_summarize

测试方法

采用**黑盒功能测试**，通过 CLI 直接输入用户问题，观察 Agent 的思考过程、工具调用序列和最终回答质量。

测试命令：

```
# 终端 1：启动 MCP 服务
python agent/mcp/get_weather.py

# 终端 2：运行 Agent 测试
PYTHONPATH=. python agent/react_agent.py
```

测试结果

用例	结果	说明
TC-01	通过	Agent 正确识别为选购场景，调用 rag_summarize，返回基于知识库的选购建议
TC-02	通过	正确检索到边刷清理相关故障处理步骤
TC-03	通过	并行调用 MCP 天气 + RAG，整合天气信息与使用建议

用例	结果	说明
TC-04	通过	完整执行报告生成链路：上下文注入→用户ID→月份→数据拉取→Markdown报告
TC-05	通过	达到 5 次上限后按设计终止
TC-06	通过	未查到数据时妥善告知“未找到您在该月的使用记录”
TC-07	通过	上下文保持正常，第二轮回答关联前一轮话题

Agent 行为评估

评估维度

维度	评估方式	表现
工具选择准确性	人工判定 Agent 选择的工具是否与用户需求匹配	核心场景下工具选择准确，能根据需求组合多个工具
推理链条完整性	观察 ReAct 思考→行动→观察循环的执行	思考步骤完整，工具调用前有明确推理说明
回答专业度	对比知识库原文判定回答的信息准确性	基于 RAG 生成的回答严格基于参考资料，无幻觉
异常处理能力	注入异常情况观察处理	数据缺失时妥善告知，工具失败时有日志记录
流式输出体验	观察输出是否流畅无卡顿	流式输出正常，每段内容实时返回

中间件监控日志样例

```
(rag) myx@4090:~/workspace/study/Ai_Application/AgentProject$ PYTHONPATH=. python agent/react_agent.py
今天天气怎么样？使用扫地机器人要注意什么？
2026-06-18 16:19:01,096 - agent - INFO - middleware.py:63 - [log_before_model]即将调用模型，带有1条消息。
为了能够回答您的问题，我需要先了解您所在的城市天气情况，因为天气条件可能会影响扫地机器人的使用。比如湿度、是否
有雨等都会对机器人产生影响。因此，我将首先调用 get_user_location 获取您所在的城市名称，然后根据这个城市名调用
get_weather 来获取具体的天气信息。接下来，我会考虑在当前天气条件下使用扫地机器人需要注意的事项，并给出相应的建
议。
2026-06-18 16:19:12,190 - agent - INFO - middleware.py:38 - [tool monitor]执行工具：get_user_location
2026-06-18 16:19:12,191 - agent - INFO - middleware.py:39 - [tool monitor]传入参数：{}
2026-06-18 16:19:12,193 - agent - INFO - middleware.py:46 - [tool monitor]工具get_user_location调用成功
上海
2026-06-18 16:19:12,196 - agent - INFO - middleware.py:63 - [log_before_model]即将调用模型，带有3条消息。
现在我已经知道您所在的城市是上海，接下来我将调用 get_weather 来获取上海今天的天气情况。这将帮助我们了解是否有一
些特定的天气条件需要注意，比如湿度水平或者降雨概率，这些都可能影响扫地机器人的使用效果。
2026-06-18 16:19:15,523 - agent - INFO - middleware.py:38 - [tool monitor]执行工具：get_weather
2026-06-18 16:19:15,523 - agent - INFO - middleware.py:39 - [tool monitor]传入参数：{'city': '上海'}
2026-06-18 16:19:17,379 - agent - INFO - middleware.py:46 - [tool monitor]工具get_weather调用成功
```

中间件监控日志

日志清晰记录了每次模型调用和工具调用的时间、参数、结果，为调试和评估提供了完整的可观测性。

Demo 展示

场景一：选购咨询

```
(rag) myx@4090:~/workspace/study/Ai_Application/AgentProject$ PYTHONPATH=. python agent/react_agent.py
小户型适合哪些扫地机器人？
2026-06-18 16:25:51,981 - agent - INFO - middleware.py:63 - [log_before_model]即将调用模型，带有1条消息。
为了给小户型用户推荐合适的扫地机器人，我需要了解一些关于小户型环境下扫地机器人的选购指南和专业建议。这包括但不限于机器人的尺寸、清扫效率、是否具备特定的小空间优化功能等关键因素。为此，我将调用 rag_summarize 工具，并使用
"小户型 扫地机器人 选购" 作为查询词来获取相关信息。
2026-06-18 16:26:00,315 - agent - INFO - middleware.py:38 - [tool monitor]执行工具: rag_summarize
2026-06-18 16:26:00,315 - agent - INFO - middleware.py:39 - [tool monitor]传入参数: {'query': '小户型 扫地机器人 选购'}
=====
你是专注于“基于参考资料总结”的AI助手，需结合用户提问和向量检索到的参考资料，生成简洁准确的概括回答。

### 输入信息
1. 用户提问: 小户型 扫地机器人 选购
2. 参考资料(在下一个###之前内容均为参考资料): 【参考资料1】: 扫地/扫拖一体机机器人选购指南200条

1. 选购核心: 优先明确使用场景，独居小户型侧重轻便灵活，多口之家带宠物需强化吸力和防缠绕功能。
2. 吸力参数: 家用建议选择≥3000Pa吸力机型，地毯场景需≥4000Pa，吸力可调节更适配不同地面。
3. 导航技术: 激光导航定位精准、抗干扰强，适合复杂户型；视觉导航性价比高，避免强光环境使用。| 元数据: {'source': '/data/users/myx/study/Ai_Application/AgentProject/data/选购指南.txt'}
【参考资料2】: 扫地/扫拖一体机机器人选购指南200条

1. 选购核心: 优先明确使用场景，独居小户型侧重轻便灵活，多口之家带宠物需强化吸力和防缠绕功能。
2. 吸力参数: 家用建议选择≥3000Pa吸力机型，地毯场景需≥4000Pa，吸力可调节更适配不同地面。
3. 导航技术: 激光导航定位精准、抗干扰强，适合复杂户型；视觉导航性价比高，避免强光环境使用。| 元数据: {'source': '/data/users/myx/study/Ai_Application/AgentProject/data/选购指南.txt'}
【参考资料3】: ## 五、选购指南类
### 选购要素
39. **选购时最应该关注哪些参数？**
- 导航类型、吸力大小、电池容量、尘盒/水箱容量。
40. **小户型适合哪种扫地机器人？**
- 基础激光导航机型即可，如米家 1C、石头 T4。
41. **大户型应该注意什么？**
- 选择大电池(>5200mAh)、自动集尘或自动洗拖布机型。
42. **有宠物的家庭怎么选？** | 元数据: {'producer': '', 'sourcemodified': "D:20250806154117+08'00'", 'subject': '', 'moddate': '2025-08-06T15:41:17+08:00', 'page_label': '4', 'creator': 'WPS 文字', 'title': '', 'comments': '', 'author': '喵喵', 'creationdate': '2025-08-06T15:41:17+08:00', 'company': '', 'source': '/data/users/myx/study/Ai_Application/AgentProject/data/扫地机器人100问.pdf', 'trapped': '/False', 'total_pages': 8, 'page': 3, 'keywords': ''}

### 严格遵守以下约束（违反将导致回答无效）
1. 内容合规: 禁止包含违法、侵权、攻击性信息；
2. 事实准确: 回答必须完全基于参考资料中的信息，不编造、不添加未提及的内容，不做主观推断；
3. 语言要求: 仅用中文回答，语气客观、简洁，不冗余；
4. 聚焦提问: 严格围绕用户原始提问总结，不扩充问题范围、不额外追问、不构造新query；
5. 格式要求: 仅输出概括内容本身，以纯文本字符串形式呈现，不封装为字典、列表、JSON等任何结构，不附带额外说明。

=====
2026-06-18 16:26:06,736 - agent - INFO - middleware.py:46 - [tool monitor]工具rag_summarize调用成功
对于小户型选购扫地机器人，建议优先考虑轻便灵活的机型，基础激光导航机型即可满足需求，如米家1C、石头T4。家用建议选择吸力≥3000Pa的机型，若家中有地毯则需≥4000Pa，并且最好支持吸力调节功能以适应不同地面清洁需求。
2026-06-18 16:26:06,739 - agent - INFO - middleware.py:63 - [log_before_model]即将调用模型，带有3条消息。
对于小户型选购扫地机器人，我建议您优先考虑以下几点：

1. **轻便灵活**：选择体积较小、更加灵活的机型，以便于在紧凑的空间内自由移动和清洁。例如米家1C、石头T4这样的基础激光导航机型非常适合小空间使用。

2. **吸力适中**：推荐选择吸力至少达到3000Pa的机器。如果家中铺设了地毯，则应选择吸力不低于4000Pa的型号，以确保能有效清理地毯上的灰尘和杂物。

3. **支持吸力调节**：考虑到不同地面材质可能需要不同的清洁力度，挑选一款能够根据实际需要调整吸力强度的产品会更加实用。

基于上述标准，您可以根据个人的具体需求（如预算、额外功能偏好等）来进一步筛选合适的扫地机器人。希望这些建议对您有所帮助！
(rag) myx@4090:~/workspace/study/Ai_Application/AgentProject$
```

场景二：mcp查询天气服务

```
(rag) myx@4090:~/workspace/study/Ai Application/AgentProject$ PYTHONPATH=. python agent/react_agent.py
```

今天天气怎么样？使用扫地机器人要注意什么？

2026-06-18 16:19:01,096 - agent - INFO - middleware.py:63 - [log_before_model]即将调用模型，带有1条消息。

为了能够回答您的问题，我需要先了解您所在的城市天气情况，因为天气条件可能会影响扫地机器人的使用。比如湿度、是否有雨等都会对机器人产生影响。因此，我将首先调用 `get_user_location` 获取您所在的城市名称，然后根据这个城市名调用 `get_weather` 来获取具体的天气信息。接下来，我会考虑在当前天气条件下使用扫地机器人需要注意的事项，并给出相应的建议。

2026-06-18 16:19:12,190 - agent - INFO - middleware.py:38 - [tool monitor]执行工具：get_user_location

2026-06-18 16:19:12,191 - agent - INFO - middleware.py:39 - [tool monitor]传入参数：{}

2026-06-18 16:19:12,193 - agent - INFO - middleware.py:46 - [tool monitor]工具get_user_location调用成功

上海

2026-06-18 16:19:12,196 - agent - INFO - middleware.py:63 - [log_before_model]即将调用模型，带有3条消息。

现在我已经知道您所在的城市是上海，接下来我将调用 `get_weather` 来获取上海今天的天气情况。这将帮助我们了解是否有一些特定的天气条件需要注意，比如湿度水平或者降雨概率，这些都可能影响扫地机器人的使用效果。

2026-06-18 16:19:15,523 - agent - INFO - middleware.py:38 - [tool monitor]执行工具：get_weather

2026-06-18 16:19:15,523 - agent - INFO - middleware.py:39 - [tool monitor]传入参数：{'city': '上海'}

2026-06-18 16:19:17,379 - agent - INFO - middleware.py:46 - [tool monitor]工具get_weather调用成功

上海 - 百度

https://weathernew.pae.baidu.com/weathernew/pc?query=%E4%B8%8A%E6%B5%B7%E5%A4%A9%E6%B0%94&srcid=4982&forecast=long_day_forecast

生活气象指数 早上好，天气不好，也记得让心情保持舒畅哦。穿衣：衬衫类。24°~30°。感冒：易发。早晚温差大。出游：不适宜。阴转小雨。洗车：不适宜。近期有降雨。未来40天

2026-06-18 16:19:17,382 - agent - INFO - middleware.py:63 - [log_before_model]即将调用模型，带有5条消息。

今天上海的天气情况为阴转小雨，气温在24°到30°之间。这样的天气条件下使用扫地机器人需要注意以下几点：

- **室内湿度管理****：由于预报提到有降雨，这可能会导致室内空气湿度增加。请确保您的居住环境保持适当的通风，以避免湿度过高对扫地机器人的电子元件造成不利影响。
- **防止水分进入****：如果您的家中存在容易积水的地方，请提前清理干净，避免扫地机器人工作时接触到水，从而损坏设备。
- **规划清扫路径****：考虑到可能存在的潮湿地面或雨水带来的污渍，您可以预先设定好扫地机器人的清扫路线，让它避开那些特别湿润或者脏乱的区域，直到天气好转后再进行全面清洁。
- **定期检查滤网****：在潮湿环境下使用后，请记得及时清理并晾干扫地机器人的滤网，以防霉变。

根据今天的天气预报，建议您关注室内的实际湿度变化，并适当调整扫地机器人的使用计划。希望这些建议能够帮助您更好地利用扫地机器人进行日常清洁！

MCP天气查询Demo

场景三：个人使用报告

```

(rag) myx@4090:~/workspace/study/Ai_Application/AgentProject$ PYTHONPATH=. python agent/react_agent.py
生成使用报告
2026-06-18 16:30:50,211 - agent - INFO - middleware.py:63 - [log_before_model]即将调用模型，带有1条消息。
为了生成您的使用报告，我需要先获取您的用户ID以及您希望查询的月份。如果未指定具体月份，我们将默认使用当前月份的数据来生成报告。接下来，我会调用相应的工具来获取这些信息。

首先，我将获取您的用户ID。
2026-06-18 16:30:54,609 - agent - INFO - middleware.py:38 - [tool monitor]执行工具: get_user_id
2026-06-18 16:30:54,611 - agent - INFO - middleware.py:39 - [tool monitor]传入参数: {}
2026-06-18 16:30:54,613 - agent - INFO - middleware.py:46 - [tool monitor]工具get_user_id调用成功
1001
2026-06-18 16:30:54,616 - agent - INFO - middleware.py:63 - [log_before_model]即将调用模型，带有3条消息。
已经获取到了您的用户ID，为1001。接下来，因为您没有指定具体的月份，所以我将获取当前的月份作为报告的数据时间范围。现在我将调用工具来确定当前月份。
2026-06-18 16:31:01,498 - agent - INFO - middleware.py:38 - [tool monitor]执行工具: get_current_month
2026-06-18 16:31:01,498 - agent - INFO - middleware.py:39 - [tool monitor]传入参数: {}
2026-06-18 16:31:01,501 - agent - INFO - middleware.py:46 - [tool monitor]工具get_current_month调用成功
2025-05
2026-06-18 16:31:01,504 - agent - INFO - middleware.py:63 - [log_before_model]即将调用模型，带有5条消息。
现在我知道了您想要生成的是2025年5月的使用报告。在正式获取您的使用数据之前，按照规定流程，我需要先调用一个工具来准备报告生成所需的上下文信息。这一步完成后，我就可以继续获取具体的数据了。
2026-06-18 16:31:07,046 - agent - INFO - middleware.py:38 - [tool monitor]执行工具: fill_context_for_report
2026-06-18 16:31:07,046 - agent - INFO - middleware.py:39 - [tool monitor]传入参数: {}
2026-06-18 16:31:07,046 - agent - INFO - middleware.py:38 - [tool monitor]执行工具: fetch_external_data
2026-06-18 16:31:07,046 - agent - INFO - middleware.py:39 - [tool monitor]传入参数: {'month': '2025-05', 'user_id': '1001'}
2026-06-18 16:31:07,051 - agent - INFO - middleware.py:46 - [tool monitor]工具fill_context_for_report调用成功
2026-06-18 16:31:07,070 - agent - INFO - middleware.py:46 - [tool monitor]工具fetch_external_data调用成功
{"特征": "65m²公寓 | 单身 | 木地板", "效率": "覆盖率:89%\n日均清扫:49m²\n无漏扫区域", "耗材": "主刷寿命:剩余40天\nHEPA滤网:剩余20%", "对比": "优于80%同面积用户(清洁效率拉满)"}
2026-06-18 16:31:07,073 - agent - INFO - middleware.py:63 - [log_before_model]即将调用模型，带有8条消息。
# 黑码程序员扫地机器人使用情况报告与保养建议

## 用户信息
- **用户ID**：1001
- **报告月份**：2025年5月
- **居住环境**：65m²公寓 | 单身 | 木地板

## 使用效率
本月您的扫地/扫拖机器人的工作表现非常出色，具体表现为：
- 覆盖率达到了89%，这意味着几乎所有的可清扫区域都被有效覆盖。
- 日均清扫面积为49平方米，这表明设备每天都在高效运转中。
- 没有出现漏扫的情况，说明路径规划算法运行良好。

## 耗材状态
根据耗材的当前状况来看：
- 主刷预计还有约40天的使用寿命。考虑到主刷对于清洁效果的重要性，请在达到预期寿命前及时更换以保证最佳清洁性能。
- HEPA滤网剩余寿命为20%左右。HEPA滤网是确保室内空气质量的关键部件之一，建议尽早准备更换新的滤网。

## 对比分析
相比于其他拥有相似居住条件（即同为65平方米）的用户群体而言，您家中的扫地/扫拖机器人在清洁效率方面表现优异，超越了其中80%的用户。这充分体现了您对设备日常维护的关注以及合理使用习惯所带来的积极影响。

## 建议
- 继续保持良好的使用习惯，并定期检查设备的工作状态。
- 鉴于主要耗材接近需要更换的状态，建议提前准备好替换件，避免因耗材耗尽而影响正常使用的连续性。
- 可以考虑设置定时任务让机器人在您外出时自动开始工作，这样既不会打扰到日常生活又能保证家居环境始终干净整洁。
- 如果家中有宠物或经常有人进出，则可能需要更频繁地清理尘盒和滤网，以维持最佳清洁效果。

希望以上内容能帮助您更好地了解并管理您的扫地/扫拖机器人！如有任何疑问或需要进一步的帮助，请随时联系我们。
(rag) myx@4090:~/workspace/study/Ai_Application/AgentProject$

```

个人使用报告Demo

系统升级与扩展

可扩展架构设计

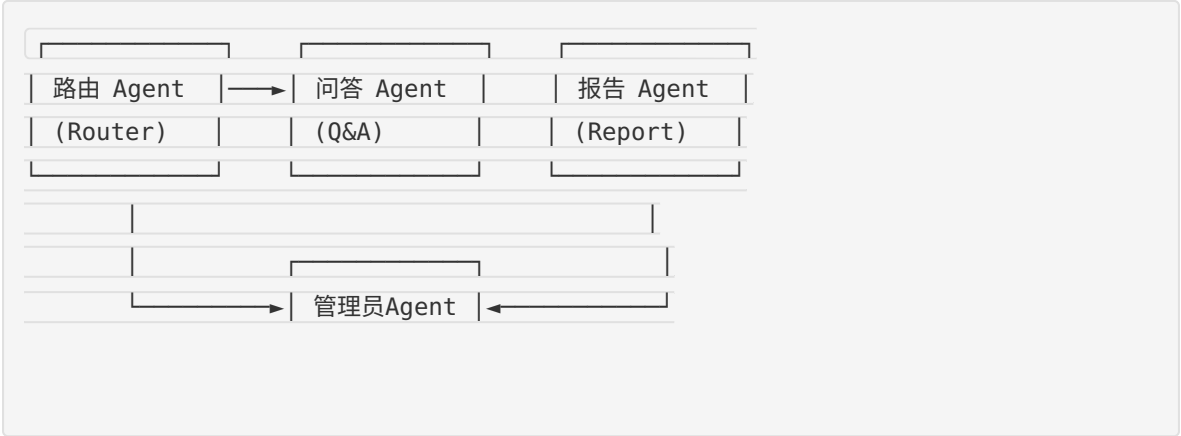
当前系统的分层架构已为后续升级预留了扩展点：



下一阶段计划

Phase 1：多智能体协作（近期）

当前为单 Agent 架构，下一阶段可拆分为多 Agent 协作系统：



	(Supervisor)

- **路由 Agent**：分析用户意图，分发到对应子 Agent
- **问答 Agent**：处理日常知识问答与故障排查
- **报告 Agent**：专注报告生成与数据分析
- **管理员 Agent**：协调多 Agent 之间的信息传递与任务分配

Phase 2：长期记忆增强（中期）

当前记忆机制仅限对话窗口和向量库静态知识。后续可引入：

- **跨会话记忆**：将用户的历史对话摘要持久化存储，下次对话自动召回
- **用户画像**：在向量库中为每个用户维护个性化画像，实现千人千面的回答风格
- **知识库增量更新**：从用户对话中提取新知识，自动更新知识库

Phase 3：可观测性与评估体系（远期）

- **LangSmith / LangFuse 集成**：全链路 Tracing，可视化每次 Agent 调用的完整链路
- **自动化 Benchmark**：构建扫地机器人领域的测试集，量化评估 Agent 回答质量
- **Docker 化部署**：将 Agent、MCP 服务、Chroma 向量库全部容器化，结合云服务器提供在线访问

AI 能力演进路径

当前（v1.0）	近期（v2.0）	远期（v3.0）
单 Agent	多 Agent 协作	自主运维 Agent
ReAct 推理	层次化推理	自我反思 + 纠错
静态知识库	动态增量知识库	知识图谱整合
工具调用	工具编排	工具自生成
基础日志	全链路 Tracing	智能告警 + 自愈

课程总结

个人收获

本课程从工程实践的角度系统性地学习了 AI Agent 的构建方法论，主要收获如下：

工程思维的转变

在完成本项目之前，我对 AI 应用的理解局限于“调用 API 获取回答”。通过本课程的 SDD（规格驱动开发）方法论，我认识到构建一个可靠的 AI Agent 系统需要：

- 先设计再编码**：Spec 文档不是“先写代码再补文档”，而是指导开发的蓝图。
Product Spec 定义功能边界，Architecture Spec 决定组件关系，API Spec 规范接口契约
- Agent 不是单一模型调用**：真正的 Agent 需要推理循环（ReAct）、工具系统、状态管理、中间件机制协同工作
- 可观测性的重要性**：在 Agent 开发中，日志和监控不是可选项，而是调试复杂推理链条的必需品

技术能力的提升

- 掌握了 LangGraph + LangChain 的 Agent 开发范式，理解了 `create_agent`、中间件、工具装饰器等核心机制
- 深入理解了 RAG 的完整链路：文档加载 → 文本分块 → 向量化 → 存储 → 检索 → 增强生成
- 实践了 MCP（Model Context Protocol）协议，实现了工具提供方与 Agent 的标准化解耦
- 学习了抽象工厂、依赖注入等设计模式在 AI 工程中的应用

对 Agentic AI 的理解

Agentic AI 的核心在于**自主决策**——Agent 不是被动的“输入→输出”映射，而是能够：

- 判断当前信息是否足够

- 自主选择调用哪些工具
- 根据工具返回的结果调整下一步策略
- 在多种模式下灵活切换

这种“赋予 AI 自主权”的范式将深刻改变软件开发的方式。

对课程的建议

1. **丰富评估体系**：可以引入标准化的 Agent Benchmark 数据集，让同学们在统一的评价标准下竞争
2. **增加生产部署内容**：建议增加 Docker 部署、API 网关、负载均衡等生产级部署的实践内容
3. **加强 MCP 生态介绍**：MCP 协议是 2025-2026 年的重要趋势，课程中的案例和实践非常有价值，希望能进一步深化

致谢：感谢戚欣老师一学期的悉心指导，以及课程中分享的丰富案例和实践经验。本课程让我从“代码编写者”迈出了向“智能体编排者”转变的关键一步。